

AI Knowledge Base for RAG

A synthetic technical knowledge document created for our first Retrieval-Augmented Generation project.

1. What is Retrieval-Augmented Generation?

Retrieval-Augmented Generation, commonly called RAG, is an architecture that combines information retrieval with text generation. Instead of asking a language model to answer a question only from information contained in its trained parameters, a RAG system first retrieves relevant information from an external knowledge source and supplies that information to the language model as context.

A typical RAG flow contains five major stages: document ingestion, text chunking, embedding generation, vector retrieval, and answer generation. The retriever identifies useful passages, while the generator uses those passages to produce a natural-language response.

2. Document Ingestion

Document ingestion is the process of converting source files into machine-readable text. A system may ingest PDFs, Markdown files, text files, HTML pages, spreadsheets, or other structured and unstructured sources.

For a PDF, the first step is usually extracting its text page by page. Metadata such as the filename, page number, document title, and section can also be retained. Preserving metadata is useful because the RAG application can later show users where an answer came from.

3. Text Chunking

Large documents are normally divided into smaller pieces called chunks before embeddings are generated. A chunk should contain enough information to represent a useful idea but should not be unnecessarily large.

A common starting strategy is a chunk size of roughly 500 to 1,000 tokens with some overlap between neighboring chunks. Overlap helps preserve context when an important sentence occurs near a chunk boundary. Chunking strategy should ultimately be evaluated against the type of documents and questions the application receives.

4. Embeddings

An embedding is a numerical vector representation of text. Texts with similar meanings tend to have vectors that are close to each other according to a chosen similarity measure.

For example, the questions 'How does document retrieval work?' and 'What is the purpose of retrieving documents?' may use different words but can have similar semantic representations. This allows an embedding-based retriever to find relevant passages even when the query does not contain the exact keywords found in the source document.

5. Vector Search

After generating embeddings for document chunks, the vectors are stored in a vector index or vector database. When a user asks a question, the question is converted into an embedding using the same embedding model. The system then searches for document vectors that are most similar to the query

vector.

FAISS is a library designed for efficient similarity search over dense vectors. It is useful for learning and for smaller applications because it can run locally without requiring a separate database service.

6. Retrieval

Retrieval is the stage where the system selects the most relevant chunks for a user question. A simple implementation may retrieve the top 3 or top 5 chunks based on vector similarity.

Retrieval quality is critical. If the correct information is not retrieved, the language model may not have enough evidence to answer correctly. Improving retrieval can involve better chunking, better embedding models, metadata filtering, query rewriting, reranking, or hybrid keyword-and-vector search.

7. Generation

In the generation stage, the retrieved chunks are placed into a prompt together with the user's question. The language model is instructed to answer using the supplied context. A well-designed prompt should discourage unsupported claims and should make the model state when the provided context is insufficient.

A basic RAG prompt can contain three parts: instructions for the model, the retrieved context, and the user's question. The application can also ask the model to return source identifiers so that the final answer can be connected back to the original documents.

8. Example RAG Request

Stage	Example
User question	What is the purpose of chunk overlap?
Retrieved context	Overlap helps preserve context when important information occurs near chunk boundaries.
Generator instruction	Answer using only the supplied context.
Final answer	Chunk overlap helps preserve context across chunk boundaries.

9. Common RAG Failure Modes

- **Poor chunking:** Important context may be split across chunks or unrelated information may be combined.
- **Weak retrieval:** The correct passage may not appear in the top results.
- **Embedding mismatch:** The embedding model may perform poorly for the application's language or domain.
- **Too much context:** Supplying many irrelevant chunks can make generation less precise.
- **Unsupported answers:** A model may produce an answer that sounds plausible even when the retrieved context does not support it.
- **Missing source tracking:** Without metadata, it becomes difficult to explain where an answer came from.

10. RAG Evaluation

A RAG application should be evaluated separately for retrieval and generation. Retrieval evaluation asks whether the correct source chunk was found. Generation evaluation asks whether the final answer is accurate, relevant, grounded in the retrieved context, and useful to the user.

For our first project, we will create a small test set of questions with expected source sections. This will let us inspect retrieval results before we focus on improving the final language-model response.

11. Project Goal

The goal of this project is to build a working RAG system from the ground up. We will first implement the individual stages ourselves so that the data flow is clear. After the basic system works, we can introduce production-oriented components and compare our implementation with frameworks such as LangChain or LlamaIndex.